Atividade Avaliativa A2 - App de Agendamento de Monitoria

Alunos: Eduardo Haag Piassa da Silva (a2669463) e Gabriel Mancuso Bonfim (a2669498)

---

1. Projeto Inicial (Passo 5)

História Escolhida (Prioridade Alta): Como Aluno, eu quero ver os horários disponíveis na monitoria de uma disciplina para reservar um atendimento.

Mapeamento de Classes e Responsabilidades

Classes de Entidade:

- **`Aluno`**: Representa o estudante que solicita a monitoria (`nome`, `ra`).
- **`Monitor`**: Representa o aluno monitor responsável pelo atendimento (`nome`, `ra`).
- **`Horario`**: Define a disponibilidade oferecida pelo monitor (`monitor`, `inicio`, `duracao_min = 30`, `reservado`).
- **`MonitoriaMateria`**: Associa uma disciplina ao seu monitor e à lista de horários oferecidos (`disciplina`, `monitor`, `horarios`).
- **`Reserva`**: Registra o agendamento efetuado (`aluno`, `disciplina`, `horario`, `topico`).

Classe de Serviço (`SistemaMonitoria`): Classe principal que gere o catálogo de monitorias e a lista de reservas do sistema (`monitorias`, `reservas`).

- `ListarDisciplinas()`: Retorna a lista de nomes de disciplinas que possuem monitorias cadastradas.
- `HorariosDisponiveis(nome_da_materia)`: Consulta e retorna os horários livres (não reservados) para a disciplina informada.
- `ReservarHorario(aluno, nome_da_materia, horario_da_reserva, topico_da_duvida)`: Valida os dados, gera uma instância de `Reserva` com duração fixa de 30 minutos, atualiza o status do horário para reservado e adiciona ao sistema.

---

2. Relatório de Execução (README)

Instruções para Executar o Sistema e os Testes

- **Versão do Python:** Python 3.10 ou superior.
- **Bibliotecas Necessárias:** Apenas bibliotecas nativas do Python (`unittest` e `datetime`). Não é necessário instalar pacotes externos.
- **Arquivo Principal (Testes):** A validação das regras de negócio é feita através do ficheiro `codigo/test_sistema_monitoria.py`.
- **Comando para executar os testes:** No terminal, a partir da pasta raiz do repositório, execute: `python -m unittest discover -s codigo` Ou diretamente no ficheiro: `python codigo/test_sistema_monitoria.py`

Plano vs. Realidade

Etapa	Data Planeada	Responsável	Realizado / Status
1. Análise e plano	16/10	Eduardo e Gabriel	Concluído no prazo.
2. Repositório e HUs	16/10 a 19/10	Eduardo (1-3) / Gabriel (4-5)	Concluído. HUs versionadas no Git.
3. Projeto inicial	19/10	Gabriel e Eduardo	Concluído. Registado antes do código.
4. Implementação	20/10	Eduardo e Gabriel	Concluído. Classes e lógicas implementadas (ver Mudanças abaixo).
5. Testes de aceitação	20/10 a 21/10	Gabriel e Eduardo	Concluído. Todos os casos a passar usando `unittest`.
6. Relatório e Doc.	21/10	Eduardo e Gabriel	Concluído. PDF unificado gerado.
7. Autoavaliação/Envio	22/10	Eduardo e Gabriel	Submissão final validada.

Atrasos e Mudanças em Relação ao Projeto Inicial

- **Classes de Serviço:** Criámos uma classe orquestradora extra chamada `SistemaMonitoria` (com auxílio de IA para a estruturação básica em Python), pois percebemos que deixar a lógica solta ou apenas nas entidades violaria a

responsabilidade única. Também introduzimos `MonitoriaMateria` para ligar adequadamente monitores, disciplinas e horários.
- **Atraso na Entrega Final:** A submissão foi realizada no dia 23/10, um dia após o prazo original planeado (22/10), o atraso ocorreu por uma pequena falta de organização da nossa parte em relação ao prazo no sistema, o que nos levou a perder o dia do envio, mesmo com a implementação já encaminhada. Assumimos a responsabilidade por este descuido no planeamento final e garantimos que todo o foco foi mantido para entregar o código e os testes totalmente operacionais conforme exigido.

---

3. Documentação Técnica

3.1 Histórias de Usuário

Com base no e-mail da Coordenação, elaboramos as seguintes histórias:

1. (Prioridade Alta) Como Aluno, eu quero ver os horários disponíveis na monitoria de uma disciplina para reservar um atendimento.

- **Critérios de aceitação:** 1) Sistema deve listar as disciplinas com monitoria disponível. 2) Exibir apenas horários disponíveis em que os monitores marcaram como livres e não reservados. 3) Reserva gerada deve ter duração fixa de 30 min. 4) Obrigatório informar o tópico da dúvida.
- **Justificativa:** Resolve o problema imediato de conflito de agendamentos e desencontros relatado no e-mail.
- **Trecho:** *"A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento. Cada atendimento dura 30 minutos..."*

2. (Prioridade Alta) Como Monitor, eu quero informar os horários que estarei disponível para atender e cancelar quando tiver provas.

- **Critérios de aceitação:** 1) Informar horários de disponibilidade. 2) Cancelar horários cadastrados. 3) Avisar alunos em caso de cancelamento.
- **Justificativa:** O aluno depende do monitor disponibilizar horários para agendar. Sem isso, o sistema base não funciona.
- **Trecho:** *"...cada monitor informa os horários em que pode atender, mas às vezes precisa cancelar por causa de prova."*

3. (Prioridade Média) Como Coordenador, eu quero um relatório mensal com as horas atendidas por cada monitor para o pagamento da bolsa, mas não pode ficar visível para todos.

- **Critérios de aceitação:** 1) Sistema deve contabilizar horas por monitor e gerar relatório. 2) Relatório disponível apenas para perfis de coordenação. 3) Atualização mensal automática.
- **Justificativa:** Envolve pagamentos (bolsa) e tem restrições de privacidade críticas.
- **Trecho:** *"A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa. Essas informações de bolsa não podem ficar visíveis para todo mundo."*

4. (Prioridade Baixa) Como Monitor, eu quero ver os tópicos de dúvida dos alunos que reservaram e abrir um atendimento em grupo para atender de uma só vez quem tem a mesma dúvida.

- **Critérios de aceitação:** 1) Monitor visualiza o tópico de cada reserva. 2) Sistema destaca dúvidas semelhantes. 3) Monitor pode alterar a reserva individual para grupo com limite de vagas.
- **Justificativa:** Otimiza o tempo, mas as dúvidas já estariam a ser respondidas individualmente. É uma melhoria de usabilidade secundária.
- **Trecho:** *"...mas alguns monitores preferem atender em grupo, quando várias pessoas têm a mesma dúvida."*

5. (Prioridade Média/Baixa) Como Professor, eu quero saber as principais dúvidas dos alunos para reforçar em aula.

- **Critérios de aceitação:** 1) Relatório lista tópicos do mais para o menos citado. 2) Acesso restrito às disciplinas que o professor leciona. 3) Filtro por período.
- **Justificativa:** Não é o núcleo de agendamentos, mas traz valor educacional elevado conforme pedido.
- **Trecho:** *"Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula."*

3.2 Testes de Aceitação (Para História 1)

Critério de Aceitação	Testes de Sucesso	Testes de Falha / Exceção	Justificativa (Não cobertos)
C1: Listar disciplinas com monitoria disponível	`test_listar_disciplinas_com_monitoria_retorna_lista_correta`	`test_listar_disciplinas_sem_monitorias_cadastradas_retorna_lista_vazia`	N/A (Totalmente coberto)
C2: Exibir apenas horários disponíveis e livres	`test_exibir_apenas_horarios_nao_reservados`	`test_consultar_horarios_disciplina_inexistente_lanca_excecao`	N/A (Totalmente coberto)
C3: Reserva com duração fixa de 30 min	`test_reserva_gerada_com_duracao_30min`	-	Não há teste de exceção pois a duração é fixa no código (parâmetro padrão `duracao_min=30`), o utilizador não consegue enviar uma duração diferente para falhar.
C4: Informar o tópico da dúvida	`test_reservar_horario_com_topico_valido_sucesso`	`test_reservar_horario_sem_topico_lanca_excecao`	N/A (Totalmente coberto)